SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM087
Name: Pranav Gajanan Revankar
Division: B
Date: 11 October 2026
GitHub / Portfolio Link (optional): https://github.com/pranavrevankar33-ctrl

---

PART A: Architecture Decision Record (ADR)

Title: Use of Breadth-First Search (FIFO Queue + Visited Set) as the Primary 8-Puzzle Solving Strategy
Status: Accepted
Date: 11 October 2026
PRN & Name: 25UAM087 — Pranav Gajanan Revankar

1. Context

In SLE-2 and SLE-3 I built an 8-Puzzle Solver System that takes a fixed start state and must reach the goal state through search. I had to decide which of my two implemented uninformed algorithms, BFS or DFS, should be the system's primary solving strategy, because they behave very differently on the same puzzle: BFS explores level by level and is guaranteed to find the goal at the shallowest depth, while DFS commits to one branch and may pass through a large part of the state space before reaching the goal. This decision affects how the Frontier and the Memory / Visited Set inside the Search Engine container (SLE-3, C4 Level 3) are used, and which algorithm's result the Output Module should be built around.

2. Decision

I decided to use Breadth-First Search as the primary solver, using a FIFO queue (collections.deque) as the Frontier and a visited set to stop states being expanded twice. DFS (a LIFO stack) is kept only as a baseline for comparison, and both algorithms share the same Neighbour Generator, get_neighbors().

3. Alternatives Considered

- **Alternative 1: Depth-First Search (stack).** Implemented and profiled in SLE-2. Rejected as the primary solver because it expanded about 21.5x more nodes and took about 27.4x longer, and it gives no guarantee of the shortest solution.
- **Alternative 2: A* with Manhattan-distance heuristic.** Considered but not implemented. It would likely expand far fewer nodes than BFS while staying optimal, but it needs a priority queue and a new Heuristic component beyond the BFS-vs-DFS comparison built for this project. Kept as a future upgrade (noted in both my SLE-2 conclusion and SLE-3 design decisions).
- **Alternative 3 (optional): Iterative Deepening DFS.** Considered only, not implemented. It combines DFS's low memory use with BFS-like completeness, but it re-expands nodes across depth limits and adds complexity this project did not need.

4. Rationale / Justification

- **BFS was far faster:** 12.494 ms against 342.052 ms for DFS, about 27.4x faster (SLE-2, Table 1).
- **BFS expanded far fewer nodes:** 6,027 against 129,832 for DFS, about 21.5x fewer, touching only 3.3% of all 181,440 reachable states against DFS's 71.6% (SLE-2, Table 1 and Fig. 3).

- **Optimality:** BFS guarantees the shortest solution when every move costs the same, matching the theory discussed in SLE-2, Section 4; DFS gives no such guarantee and its cost depends on move order and start state.
- **The time gap follows the node gap, not per-step cost:** time per expanded node was similar for both (about 2.1 microseconds for BFS, 2.6 for DFS), so DFS was slower mainly because it expanded far more nodes (SLE-2, Section 4).
- **Fits the SLE-3 design:** BFS and DFS differ only in the Frontier container (deque vs list); the Neighbour Generator and Memory / Visited Set containers need no change at all (SLE-3, Fig. 2 and Fig. 3).

5. Consequences

Positive:

- Always returns the solution in the fewest moves on this puzzle (optimal for equal-cost moves).
- The design stays simple: only the Frontier's data structure changes between BFS and DFS, so no extra container was needed in SLE-3.
- Results are fully reproducible: node counts are identical on every run, since both the puzzle and the algorithms are deterministic.

Negative / Trade-offs:

- BFS must keep every visited state in memory, so memory (not time) would be the first limit on a larger puzzle such as the 15-puzzle.
- BFS is uninformed: it ignores how close a state actually is to the goal, so A* with a heuristic would likely expand even fewer nodes.
- These numbers come from one start state and one fixed move order, so they should not be read as a universal result for every 8-puzzle instance.

6. References (optional)

- GitHub profile: https://github.com/pranavrevankar33-ctrl
- SLE-2 Profiling Report: Table 1 (Results), Fig. 2 (flame graph) and Fig. 3 (time/nodes chart), Section 4 (Justification & Analysis); code in bfs_dfs_8puzzle.py.
- SLE-3 C4 Report: Fig. 2 (Container diagram), Fig. 3 (Component diagram of the Search Engine) and Section 6 (Design Decisions).

PART B: Short Summary of My SLE Journey

SLE-1: I built a simple rule-based AI chatbot (agent.py) using if-elif-else statements, with the help of GitHub Copilot Chat, and kept an AI Contribution Log describing what the AI generated and what I tested myself. SLE-2: I moved to search and profiled BFS vs DFS on the same 8-puzzle using time.perf_counter, a manual node counter, and a cProfile-based flame graph, with 3 runs per algorithm; BFS was about 27.4x faster and expanded about 21.5x fewer nodes than DFS. SLE-3: I documented the system using the four C4 levels (Context, Container, Component, Code) and saw that BFS and DFS differ only in how the Frontier is implemented. SLE-4: this ADR records my decision to adopt BFS as the system's primary strategy, based directly on the SLE-2 numbers and the SLE-3 design.

PART C: AI Contribution Note

- **AI tools used:** Gemini and ChatGPT (SLE-1 and the first draft of the SLE-2 code), and Claude (SLE-2 report formatting and flame graph, SLE-3 diagrams, and this ADR).
- **What AI helped with:** arranging this ADR in the guideline's exact format, drafting the wording of the Context and Rationale sections from my SLE-1 to SLE-3 reports, and

recalculating the ratios quoted above (27.4x, 21.5x, 3.3%/71.6%) directly from my SLE-2 table.

- **What I did myself:** chose the decision (BFS as the primary strategy), checked every number against my own SLE-2 report and every container/component name against my SLE-3 diagrams and code, and prepared my own viva answers below.

PART D: Viva Preparation Notes

- **Why BFS, and why not the others?** BFS is complete and optimal for equal-cost moves, so it is guaranteed to find the fewest-move solution. DFS expanded about 21.5x more nodes and gave no such guarantee. A* would likely do even better than BFS but needs a heuristic and a priority queue I did not implement in this project.
- **Profiling numbers to quote (BFS / DFS):** Time 12.494 / 342.052 ms. Nodes 6,027 / 129,832. Share of all 181,440 reachable states: 3.3% / 71.6%.
- **C4 Container diagram in under a minute:** Input Module holds the fixed start/goal states; the Experiment Driver runs each algorithm 3 times and times every run; the Search Engine runs BFS or DFS using the shared Neighbour Generator; the Memory / Visited Set stops states being expanded twice; the Output Module prints the final time and node-count summary. Only the Frontier differs between BFS and DFS.
- **If the problem becomes larger:** on something like the 15-puzzle, the number of reachable states grows exponentially, so BFS's memory use (storing every visited state) becomes the real bottleneck, not just time. At that scale I would move to A* with a Manhattan-distance heuristic.
- **What AI generated and what I changed:** Gemini/ChatGPT drafted the original get_neighbors/solve_bfs/solve_dfs functions; I ran the code myself, verified the node counts matched exactly, and wrote the header details, the AI notes, and this ADR's decision and justification in my own words.